

Airflow Install

There will be 3 terminals

- **Terminal 1** — setup, then left idle
 - **Terminal 2** — runs `airflow standalone` (long-running, stays open)
 - **Terminal 3** — HDFS checks, DAG creation, triggering, verification
-

1. Prerequisites (Terminal 1)

```
sudo apt update
sudo apt install -y \
    python3 python3-pip python3-venv python3-dev \
    build-essential libssl-dev libffi-dev libpq-dev \
    curl wget git unzip
python3 --version
```

`python3-venv` is included in this install list on purpose — without it, `python3.12 -m venv` fails with `ensurepip is not available`. If you ever hit that error separately, run `sudo apt install -y python3.12-venv`.

```
java -version
hdfs version
which hdfs
```

```
siya@siya:~$ java -version
hdfs version
which hdfs
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-24.04.2-Ubuntu, mixed mode, sharing)
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /opt/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
/opt/hadoop/bin/hdfs
```

2. Create the Airflow project and venv (Terminal 1)

```
mkdir -p ~/airflow-project
cd ~/airflow-project
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
export AIRFLOW_HOME=$(pwd)
printf 'AIRFLOW_HOME=%s\n' "$AIRFLOW_HOME"
which python
```

Expected:

```
AIRFLOW_HOME=/home/siya/airflow-project
/home/siya/airflow-project/venv/bin/python
```

3. Install Airflow 3.3.0 (Terminal 1)

Set the version variables **as one clean block**

```
export AIRFLOW_VERSION=3.3.0
export PYTHON_VERSION=$(python -c "import sys; print(f'{sys.version_info.major}.{sys.version_info.minor}')" )
export CONSTRAINT_URL="https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"
```

Verify before installing anything:

```
echo "PYTHON_VERSION=[${PYTHON_VERSION}]"
echo "CONSTRAINT_URL=[${CONSTRAINT_URL}]"
```

Expected: no stray characters, no `%0A` , no literal `{PYTHON_VERSION}` :

```
(venv) siya@siya:~/airflow-project$ echo "PYTHON_VERSION=[${PYTHON_VERSION}]"
echo "CONSTRAINT_URL=[${CONSTRAINT_URL}]"
PYTHON_VERSION=[3.12]
CONSTRAINT_URL=[https://raw.githubusercontent.com/apache/airflow/constraints-3.3.0/constraints-3.12.txt]
```

Install:

```
pip install "apache-airflow==${AIRFLOW_VERSION}" --constraint "${CONSTRAINT_URL}"
```

you may get a wheel error, its okay, next Graphviz, to remove a harmless CLI warning:

```
sudo apt install -y graphviz
pip install graphviz --constraint "${CONSTRAINT_URL}"
```

Verify:

```
airflow version
pip check
```

```
(venv) siya@siya:~/airflow-project$ airflow version
pip check
3.3.0
No broken requirements found.
(venv) siya@siya:~/airflow-project$ sudo apt install -y libkrb5-dev
```

4.HDFS provider package (Terminal 1)

```
sudo apt install -y libkrb5-dev
pip install apache-airflow-providers-apache-hdfs --constraint "${CONSTRAINT_URL}"
```

Without `libkrb5-dev` first, this fails on building the `gssapi` wheel with `krb5-config: not found`. Installing the system package and re-running fixes it cleanly.

5. Initialize the local Airflow config and DB (Terminal 1)

```
cd ~/airflow-project
source venv/bin/activate
export AIRFLOW_HOME=$(pwd)
export AIRFLOW__CORE__LOAD_EXAMPLES=False
export AIRFLOW__CORE__PARALLELISM=2
```

```
export AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG=2
export AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG=1

airflow db migrate
mkdir -p dags logs
```

6. Start Airflow (Terminal 2)

Open a **second terminal**. This process stays running for the rest of the lab.

```
cd ~/airflow-project
source venv/bin/activate
export AIRFLOW_HOME=$(pwd)
export AIRFLOW__CORE__LOAD_EXAMPLES=False
export AIRFLOW__CORE__PARALLELISM=2
export AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG=2
export AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG=1
airflow standalone
```

```
(venv) siya@siya:~/airflow-project$ cd ~/airflow-project
(venv) siya@siya:~/airflow-project$ source venv/bin/activate
(venv) siya@siya:~/airflow-project$ export AIRFLOW_HOME=$(pwd)
(venv) siya@siya:~/airflow-project$ export AIRFLOW__CORE__LOAD_EXAMPLES=False
(venv) siya@siya:~/airflow-project$ export AIRFLOW__CORE__PARALLELISM=2
(venv) siya@siya:~/airflow-project$ export AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG=2
(venv) siya@siya:~/airflow-project$ export AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG=1
(venv) siya@siya:~/airflow-project$ airflow db migrate
2026-08-10T08:49:14.742159Z [info] Performing upgrade to the metadata database [airflow.cli.commands.db_command] loc=db_command.py:134 url=sqlite:///home/siya/airflow-project/airflow.db
2026-08-10T08:49:14.882802Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.883008Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.883100Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.883522Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.883641Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.883709Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:49:14.899950Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:210
2026-08-10T08:49:14.900156Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:213
2026-08-10T08:49:15.183535Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:210
2026-08-10T08:49:15.183792Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:213
2026-08-10T08:49:15.184174Z [info] ] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:749
2026-08-10T08:49:15.184278Z [info] ] Creating global lock context [airflow.utils.db] loc=db.py:730
2026-08-10T08:49:15.184364Z [info] ] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -4 Current Checked out connections: 1 [airflow.utils.db] loc=db.py:733
2026-08-10T08:49:15.184426Z [info] ] Creating metadata [airflow.utils.db] loc=db.py:735
```

Watch for:

```
standalone | Airflow is ready
api-server | Host: 0.0.0.0:8080
```

```
scheduler | Loaded executor: :LocalExecutor:
dag-processor | Found 1 files for bundle dags-folder
```

No `sqlite3.OperationalError: database is locked` should appear anywhere.

The password get generated and you can retrieve it anytime with:

```
cat ~/airflow-project/simple_auth_manager_passwords.json.generated
```

7. HDFS pre-flight checks (Terminal 3)

Open a **third terminal**. Confirm HDFS itself is healthy *before* Airflow tries to shell out to it — this isolates Hadoop problems from Airflow problems.

```
start-dfs.sh
start-yarn.sh

hdfs version
hdfs getconf -confKey fs.defaultFS
hdfs dfsadmin -report | head -n 25
hdfs dfs -mkdir -p /user/student/cli_lab
```

Verify:

```
hdfs dfs -ls /user/student/cli_lab
hdfs dfs -ls /user/student
```

```
siya@siya:~/airflow-project$
siya@siya:~/airflow-project$ hdfs dfs -ls /user/student/cli_lab
siya@siya:~/airflow-project$ hdfs dfs -ls /user/student/
Found 1 items
drwxr-xr-x  - siya supergroup          0 2026-08-10 14:24 /user/student/cli_lab
siya@siya:~/airflow-project$
```

8. Create the DAG (Terminal 3)

```
mkdir -p ~/airflow-project/dags
nano ~/airflow-project/dags/hdfs_workflow.py
```

Paste exactly:

```
from __future__ import annotations

import pendulum

from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator

HDFS_DIR = "/user/student/cli_lab"
LOCAL_FILE = "/tmp/cli_test.csv"
HDFS_FILE = f"{HDFS_DIR}/cli_test.csv"

with DAG(
    dag_id="student_cli_lab",
    description="Generate a CSV, upload it to HDFS, and verify the content",
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    schedule=None,
    catchup=False,
    max_active_runs=1,
    tags=["hdfs", "cli-lab"],
) as dag:

    generate_data = BashOperator(
        task_id="generate_data",
        bash_command=(
            "set -euo pipefail; "
            "printf 'id,status\n1,success\n' > /tmp/cli_test.csv; "
            "echo 'Generated local file: '; cat /tmp/cli_test.csv"
        ),
    )
```

```

create_hdfs_folder = BashOperator(
    task_id="create_hdfs_folder",
    bash_command=(
        "set -euo pipefail; "
        "hdfs dfs -mkdir -p /user/student/cli_lab"
    ),
)

upload_data = BashOperator(
    task_id="upload_data",
    bash_command=(
        "set -euo pipefail; "
        "hdfs dfs -put -f /tmp/cli_test.csv "
        "/user/student/cli_lab/cli_test.csv"
    ),
)

verify_data = BashOperator(
    task_id="verify_data",
    bash_command=(
        "set -euo pipefail; "
        "hdfs dfs -test -e /user/student/cli_lab/cli_test.csv; "
        "echo 'Verified HDFS file: ';"
        "hdfs dfs -cat /user/student/cli_lab/cli_test.csv"
    ),
)

[generate_data, create_hdfs_folder] >> upload_data >> verify_data

```

Save and exit (`Ctrl+O` , `Enter` , `Ctrl+X`).

Filename vs. DAG ID: the file is `hdfs_workflow.py` , but every CLI command below uses `student_cli_lab` (the `dag_id=` value). These are not interchangeable.

9. Parse and register the DAG (Terminal 3)

```
cd ~/airflow-project
source venv/bin/activate
export AIRFLOW_HOME=$(pwd)

python dags/hdfs_workflow.py
airflow dags list-import-errors
airflow tasks list student_cli_lab
```

```
siya@siya:~/airflow-project$ cd ~/airflow-project
source venv/bin/activate
export AIRFLOW_HOME=$(pwd)
(venv) siya@siya:~/airflow-project$ python dags/hdfs_workflow.py
(venv) siya@siya:~/airflow-project$ airflow dags list-import-errors
2026-08-10T08:58:52.107128Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:58:52.108926Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:58:52.109072Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:58:52.109150Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:58:52.109221Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T08:58:52.109311Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
No data found
(venv) siya@siya:~/airflow-project$ airflow dags list | grep student_cli_lab
(venv) siya@siya:~/airflow-project$ airflow tasks list student_cli_lab
create_hdfs_folder
generate_data
upload_data
verify_data
(venv) siya@siya:~/airflow-project$
```

Then check the DAG shows up in the metadata DB — this can take up to ~30 seconds after saving the file, since the `dag-processor` (in Terminal 2) scans on a cycle, not instantly:

```
airflow dags list | grep student_cli_lab
```

If empty, wait and retry. You should eventually see:

```
(venv) siya@siya:~/airflow-project$ airflow dags list | grep student_cli_lab
student_cli_lab | /home/siya/airflow-project/dags/hdfs_workflow.py | airflow | True | dags-folder | None
```

`True` here just means "currently paused"

10. Unpause, trigger, and check run state (Terminal 3)

```
airflow dags unpause student_cli_lab
airflow dags trigger student_cli_lab
```

```
airflow dags list-runs student_cli_lab
```

Right after triggering, `state` will show `queued` or `running`

Note: the `unpause` command's own output table sometimes still shows `is_paused: True` even though it worked; trust `list-runs` /the UI over that one line.

```
(venv) siya@siya:~/airflow-project$ airflow dags unpause student_cli_lab
airflow dags trigger student_cli_lab
airflow dags list-runs student_cli_lab
2026-08-10T09:07:14.225710Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:14.236695Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:14.236851Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:14.236933Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:14.237003Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:14.237070Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
dag_id | is_paused
=====+=====
student_cli_lab | True
```

```
2026-08-10T09:07:17.684444Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.684650Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.684849Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.684965Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.685052Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.685129Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:17.790861Z [info] ] creating dag run [airflow.serialization.definitions.dag] loc=dag.py:607 logical_date=None partition_key=None run_after=datetime.datetime(2026, 8, 10, 9, 7, 17, 777729, tzinfo=Timezone('UTC')) run_id=manual__2026-08-10T09:07:17.777729+00:00
conf | dag_id | dag_run_id | data_inte | rval_star | data_inte | end_date | uling_deci | logical_d | run_type | start_date | state | triggering
=====+=====+=====+=====+=====+=====+=====+=====+=====+=====+=====+=====+=====
{} | student_c | manual__2 | None | None | None | None | None | None | manual | None | queued | siya
li_lab | 026-08-10 | T09:07:17 | | | | | | | | | |
| | .777729+0 | | | | | | | | | |
| | 0:00 | | | | | | | | | |
```

```
2026-08-10T09:07:22.885258Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:22.885469Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:22.885569Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:22.885638Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:22.885702Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:07:22.885763Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
dag_id | run_id | state | run_after | logical_date | start_date | end_date
=====+=====+=====+=====+=====+=====+=====
student_cli_lab | manual__2026-08-10T09:07:17.777729+00:00 | running | 2026-08-10T09:07:17.777729+00:00 | 2026-08-10T09:07:18.02424 | 2+00:00 |
```

Wait a few seconds (give it a minute) and re-check:

```
airflow dags list-runs student_cli_lab
```

Expected, eventually:

```
(venv) siya@siya:~/airflow-project$ airflow dags list-runs student_cli_lab
2026-08-10T09:08:36.156197Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:08:36.156468Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:08:36.156567Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:08:36.156645Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:08:36.156711Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-08-10T09:08:36.156778Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
dag_id | run_id | state | run_after | logical_date | start_date | end_date
=====+=====+=====+=====+=====+=====+=====
student_cli_lab | manual__2026-08-10T0 | success | 2026-08-10T09:07:17. | | 2026-08-10T09:07:18.0 | 2026-08-10T09:07:40.3
| 9:07:17.777729+00:00 | | 777729+00:00 | | 24242+00:00 | 23196+00:00
```

11. Verify the actual HDFS result (Terminal 3)

```
hdfs dfs -cat /user/student/cli_lab/cli_test.csv
```

Expected:

```
(venv) siya@siya:~/airflow-project$ hdfs dfs -cat /user/student/cli_lab/cli_test.csv
id,status
1,success
```

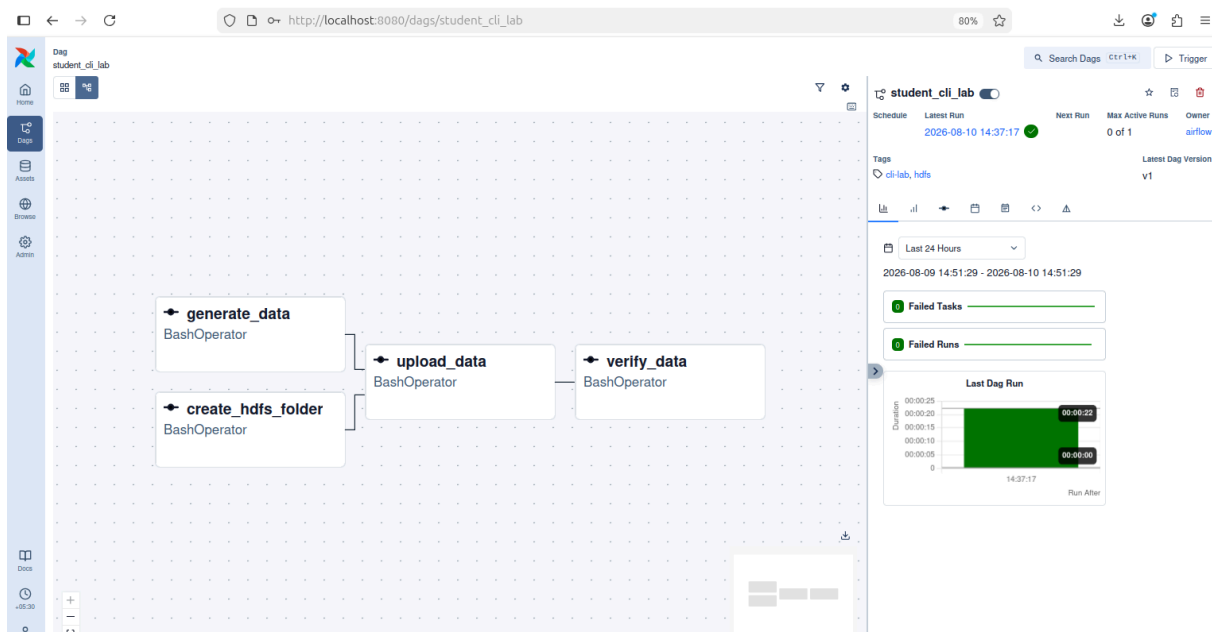
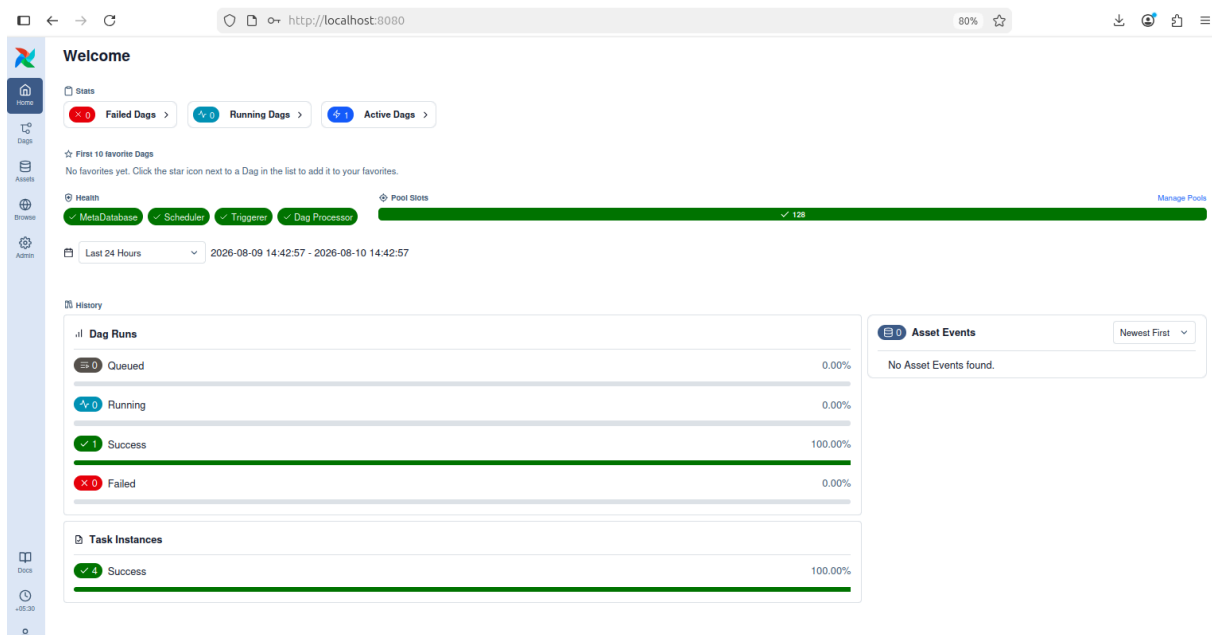
12. UI check

```
http://localhost:8080
```

Log in as `admin` with the password from:

```
cat /home/$USER/airflow-project/simple_auth_manager_passwords.json.generated
```

use these credentials and login:



Troubleshooting index

Symptom	Cause	Fix
ensurepip is not available on venv creation	python3.12-venv (or python3-venv) package missing	sudo apt install -y python3.12-venv (or python3-venv), then recreate the venv
404 Client Error with %0A%7BPYTHON_VERSION%7D in the	Constraints block copy-pasted across a	Retype the three export lines as clean single lines;

Symptom	Cause	Fix
URL	line-wrapped PDF/doc	verify with <code>echo "\$CONSTRAINT_URL"</code> before installing
<code>gssapi</code> wheel build fails: <code>krb5-config: not found</code>	<code>libkrb5-dev</code> missing, needed by the <i>optional</i> HDFS provider's Kerberos extra	<code>sudo apt install -y libkrb5-dev</code> , then retry — or just skip the HDFS provider entirely, since this DAG doesn't need it
<code>airflow dags list-runs --dag-id ...</code> or <code>-d ...</code> → full <code>airflow help</code> + <code>unrecognized arguments</code>	Airflow 3.3.0's CLI takes <code>dag_id</code> positionally, no flag	<code>airflow dags list-runs student_cli_lab</code> (no flag)
<code>airflow dags list</code>	<code>grep <id></code> returns nothing right after creating the DAG	<code>dag-processor</code> hasn't scanned the new file yet (runs on a ~30s cycle)
<code>hdfs dfs -cat</code> → <code>No such file or directory</code>	DAG hadn't actually been triggered yet (earlier trigger attempt failed silently due to the flag issue above)	Confirm <code>list-runs</code> shows <code>state: success</code> first, then re-check the file